密码系统设计 第五周预习报告

学号：20241328　**姓名**：蔡贺俊
学习内容：《Windows C/C++加密解密实战》第 7 章“数字签名技术”
课程仓库：1515720698/cryptosystem

一、AI 对学习内容的总结（1分）

> 使用的 AI 工具：OpenCode（模型：gpt-6-astra）。本报告依据课程要求、教材公开目录及 OpenSSL 官方文档整理；本次未获取教材第 7 章付费全文。RSA-PSS 参数配置与 OpenSSL 3.x 接口说明属于补充学习内容。

第 7 章的主线是：**理解数字签名解决的问题 → 理解 RSA 签名 → 用 OpenSSL 命令签名验签 → 用 C 语言调用签名接口。** 本周从第四周的 SM2 签名扩展到通用数字签名概念与 RSA 实现。

1. 7.1、7.2：数字签名的概念、性质与应用

数字签名是签名者使用私钥对特定消息产生签名数据，其他人使用对应公钥检查消息与签名是否匹配的一种密码机制。

一个签名方案通常包括三类算法：

```
KeyGen → 生成私钥 sk 和公钥 pk
Sign(sk, M) → 为消息 M 生成签名 σ
Verify(pk, M, σ) → 接受或拒绝
```

签名者保管私钥，验证者使用公钥。攻击者即使能看到公钥、消息和已有签名，也不应能够为未经授权的新消息伪造有效签名。

主要作用与边界：

- **完整性**：消息被修改后，原签名通常不能通过验证。
- **来源真实性**：在公钥身份绑定可信、私钥未泄露等前提下，签名为判断消息来源提供依据。
- **不可否认性的技术支撑**：公钥可供第三方验证，而签名私钥由签名者控制。但现实中的证据效力还取决于密钥管理、可信时间、证书状态及制度流程，不能说“验签成功就绝对无法抵赖”。
- **不提供机密性**：公开消息和签名并不会隐藏消息内容。若还需要保密，必须结合加密协议。
- **不自动防重放**：攻击者可以再次发送原消息及其有效签名；业务系统需把流水号、用途、接收方、有效期等纳入签名内容并检查。

签名与手写签名图片不同：复制一张签名图片可以粘贴到别处，而密码学签名与特定消息字节关联。修改消息后不能直接复用原签名。

分类与场景：可按算法区分 RSA、DSA、ECDSA、SM2 等；按签名组织方式区分附加式签名与消息恢复型签名；盲签名、群签名、多重签名等用于特殊需求。本周命令实验采用附加式、分离保存的签名，验签时仍需提供原消息。实际应用包括软件发布、电子合同和证书签发，具体签名对象及验证策略取决于协议。

2. 7.3：RSA 公钥算法在数字签名中的应用

对 RSA 而言，公钥包含模数 `N` 和公开指数 `e`，私钥包含私有指数 `d` 等参数。可以用下面的示意流程理解 RSA 签名，但不能省略其中的编码步骤：

```
签名：消息 M → 杂凑与规范编码 → 整数代表值 m → s = m^d mod N
验签：签名 s → m' = s^e mod N → 检查规范编码及其与消息 M 的关系
```

验签的目的不是“解密并取回完整原文”。尤其对 RSA-PSS，验证者会按规定检查编码结构、摘要和盐值关系，不能仅把某段输出与裸摘要比较。

为什么先计算杂凑？消息可能很长，而 RSA 的模运算处理长度受模数限制。杂凑把变长消息映射为固定长度摘要，使签名能够处理大文件。摘要算法还必须具有适当的抗碰撞等安全性质，不能因为输出长度合适就随意选 MD5 等弱算法。

两种常见 RSA 签名编码方案如下：

对比项	RSASSA-PKCS1-v1_5	RSASSA-PSS
基本特点	对摘要及其算法标识作规定编码	使用消息摘要、盐值与掩码生成函数构造编码
相同密钥、消息及参数重复签名	签名相同	使用随机非零长度盐时，签名通常不同
实现关注点	严格检查完整编码	摘要、MGF1 摘要、盐长度等参数需兼容
学习定位	理解既有协议和常见接口	本周补充实验采用的方案

PSS 是 **Probabilistic Signature Scheme**，MGF1 是一种**掩码生成函数。**本次统一采用 `SHA-256 + MGF1-SHA256 + 32 字节盐`。盐值不是需要另外保密的口令，也不需要单独传给验证者；验证流程根据签名恢复出的编码进行检查。

应区分：**OAEP 用于 RSA 加密，PSS 用于 RSA 签名。**二者不能互换，也都不能用“裸 RSA 模幂”代替。

3. 7.4：使用 OpenSSL 命令进行签名与验签

基本流程是生成密钥、导出公钥、对消息签名，再由公钥验证。以下命令输入文件名保持一致，长命令均可整行执行：

```
# 生成 2048 位 RSA 测试私钥并导出公钥
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out private.pem
openssl pkey -in private.pem -pubout -out public.pem

# 显式指定 PSS 参数并生成二进制签名
openssl dgst -sha256 -sign private.pem -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -s
igopt rsa_mgf1_md:sha256 -out signature.bin message.txt

# 使用同一组参数验证
openssl dgst -sha256 -verify public.pem -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -
sigopt rsa_mgf1_md:sha256 -signature signature.bin message.txt
```

- `-sign` 指定私钥；`-verify` 指定公钥。
- `-sha256` 指定对输入消息计算的摘要算法。
- `-sigopt` 设置签名算法参数，不能仅假定两端默认值相同。
- `-out` 保存签名结果，`-signature` 读取已有签名。
- 成功输出 `Verified OK`；失败应结合退出码、错误信息以及输入文件判断原因。

`dgst -sign` 接收的是消息并在内部计算摘要。不能先计算一次摘要，再不加区分地把摘要文件当作原消息传入，否则签名的对象发生变化。

签名文件是二进制数据。若为了传输进行了 Base64 或十六进制编码，验签前需要按约定解码，不能把可打印文本直接作为原签名字节。

4. 7.5：基于 OpenSSL 的签名验签编程

教材目录将本节分为“直接使用 RSA 函数”和“使用 EVP 系列函数”两条路线。低层 `RSA_sign/RSA_verify` 等接口适合帮助理解算法与摘要的关系，但这些低层 RSA 接口在 OpenSSL 3.0 起已被弃用；新程序优先使用 EVP 高层接口。

本周 C 示例使用如下调用链：

```
读取私钥 / 公钥 → EVP_MD_CTX_new
→ EVP_DigestSignInit / EVP_DigestVerifyInit
→ 显式设置 RSA-PSS、MGF1 摘要、盐长度
→ 多次 EVP_DigestSignUpdate / EVP_DigestVerifyUpdate
→ EVP_DigestSignFinal / EVP_DigestVerifyFinal
→ 释放缓冲区、摘要上下文与密钥对象
```

关键实现细节：

1. **初始化必须检查返回值。**失败后不能继续调用后续密码操作。
2. **流式处理按实际读取字节数更新。**本次示例每次读取 7 字节，用于验证分块边界不影响结果；读取模式为 `rb`。
3. **签名缓冲区先查询长度再分配。**先用 `EVP_DigestSignFinal(ctx, NULL, &len)` 查询，再分配空间并调用它写入结果。
4. **验签只接受返回值 1。** `EVP_DigestVerifyFinal` 返回 0 表示签名未通过；其他值也属于失败，可能是更严重的操作错误。不能简单写成 `if (ret)`，否则负值可能被误判为成功。
5. **明确资源所有权。**本例由初始化函数创建并交出的 `EVP_PKEY_CTX` 随 `EVP_MD_CTX` 释放，不再单独释放，避免重复释放。

完整源码见 `code/rsa_pss.c`。它不仅完成本地签名验签，还与 OpenSSL 命令行进行了双向验证。

二、对 AI 总结的反思与补充（2分）

> 本节结合本周内容整理，部分观点可在后续实验中继续核对。

1. AI 总结的问题

1. **“完整性、真实性、不可否认性”不能只背名词。**这些性质均有前提。拿到一个未知来源的公钥并验签成功，并不能直接认定签名者的现实身份；私钥泄露后，仅靠算法也不能区分真正持有者与攻击者。
2. **“对摘要签名”容易让初学者重复计算摘要。**概念上确实涉及摘要，但不同 API 的输入层次不同。`EVP_DigestSign*` 和 `dgst -sign` 自己做摘要；使用其他接收预计算摘要的接口时，又要正确配置算法与参数。
3. **“随机化签名”不等于“每一种签名都必须不同”。**本次采用随机盐的 PSS，两次签名不同；PKCS#1 v1.5 则可以相同。不能仅凭两次输出是否相同来判断所有签名算法的安全性。
4. **只有正向用例，无法说明失败路径正确。**“签名后验签成功”可能掩盖程序忽略返回值、使用错误输入或参数未受检查等问题。因此应加入篡改、错误公钥、参数不一致和不同编码的反例。

2. 自己的补充

补充一：区分普通杂凑、HMAC、数字签名。

机制	需要什么密钥	谁能生成合法结果	是否适合独立第三方验证来源
普通杂凑	不需要密钥	任何人	不能仅靠摘要确定来源
HMAC	共享秘密密钥	所有掌握共享密钥的参与方	验证者自己也能生成，不能据此区分双方责任

机制	需要什么密钥	谁能生成合法结果	是否适合独立第三方验证来源
数字签名	私钥签名、公钥验签	合法私钥持有者	支持公开验证，但还要确认公钥身份绑定

若攻击者同时替换文件和普通摘要，接收者可能发现不了。签名的额外价值在于：攻击者不能仅凭公钥就为新内容生成合法签名。公钥本身仍需通过可信途径获得。

补充二：签名对象是字节，不是人眼看到的“意思”。

`LF` 与 `CRLF`、UTF-8 与其他编码、是否包含 BOM、末尾是否有换行，都会改变签名输入。若协议要求规范化 JSON、字段顺序或文本换行，应在签名之前约定并完成；验签时不能为了让签名通过，随意猜测或修改接收到的内容。

补充三：安全验证应同时考虑上下文。

“转账 100 元”的裸文本签名不一定绑定收款人、币种或请求编号。更完整的签名对象应包含业务类型、双方标识、金额、时间或有效期以及唯一请求编号，并使用明确编码。接收方还需要检查重放与权限，签名验真不等于业务应被接受。

补充四：与前四周形成联系。

第三周的杂凑解释为什么可高效处理大消息；第二周的 DER / PEM 解释密钥与签名材料如何编码；第四周的 SM2 说明不同签名算法有不同参数，例如 SM2 的 ID / ZA；第五周进一步学习通用签名接口、RSA 签名方案与严格验签。不同算法可共用部分 EVP 调用框架，但不能照搬全部参数。

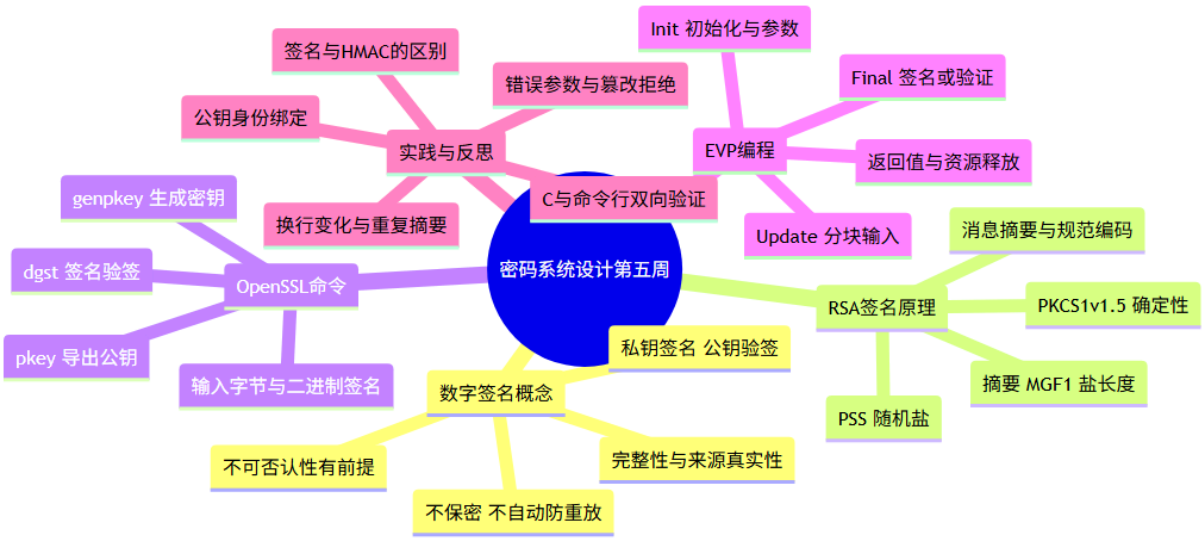
三、学习思维导图（2分）

Mermaid 代码

```
mindmap
  root((密码系统设计第五周))
    数字签名概念
      私钥签名 公钥验签
      完整性与来源真实性
      不可否认性有前提
      不保密 不自动防重放
    RSA签名原理
      消息摘要与规范编码
      PKCS1v1.5 确定性
      PSS 随机盐
      摘要 MGF1 盐长度
    OpenSSL命令
      genpkey 生成密钥
```

```
    pkey 导出公钥
    dgst 签名验签
    输入字节与二进制签名
  EVP编程
    Init 初始化与参数
    Update 分块输入
    Final 签名或验证
    返回值与资源释放
  实践与反思
    签名与HMAC的区别
    公钥身份绑定
    换行变化与重复摘要
    错误参数与篡改拒绝
    C与命令行双向验证
```

思维导图截图



以上图片由 Mermaid 源码实际渲染生成，独立源文件为 `assets/week5-mindmap.mmd`。

四、基于 AI 的学习（2分）

围绕两个不易理解的概念，采用苏格拉底式提问展开学习，从现象逐步分析原理、适用条件与实际应用。

概念一：签名与加密、HMAC 到底有什么不同

提示词：我在学习数字签名、加密和 HMAC 的区别，请你以苏格拉底的方式对我进行提问，一次一个问题，先从“公开验签为什么不会让别人伪造签名”开始。

AI 问题 1：如果所有人都能获得公钥并验证签名，为什么不能据此产生新消息的签名？

解答：验证只需要公开信息，生成合法签名则需要私钥。即使知道公钥和已有的消息—签名对，也应该很难为未经授权的新消息产生有效签名。

AI 问题 2：既然验证可以公开，数字签名会隐藏消息吗？

解答：不会。数字签名主要回答消息有没有被修改、是否与相应私钥有关，并不负责隐藏消息。如果还需要保密，就要结合加密。

AI 问题 3：HMAC 也能检测消息被修改，为什么不能直接替代所有数字签名场景？

解答：因为 HMAC 的生成和验证使用共享密钥，验证者自己也能计算合法的 HMAC。发生争议时，第三方很难只凭 HMAC 判断是哪一方生成的。数字签名则把私钥和公钥分开，其他人可以用公钥验证，却不能因此代替签名者生成签名。

AI 问题 4：别人交给你“文件、签名和一个公钥”，验签成功，是否已经证明文件来自老师？

解答：还没有，因为这个公钥也可能是对方自己生成的。还需要通过可信渠道、证书链或预先确认的公钥指纹，确定公钥确实属于老师。验签成功首先说明消息、签名和这个公钥匹配，不能直接证明现实身份。

AI 问题 5：如果原消息和原签名再次被发送，验签会不会自动失败？

解答：通常不会，因为原消息和签名都没有变。因此不能把验签当成防重放机制，还需要把请求编号、有效期等放进签名内容，并在验证端检查是否重复或过期。

AI 评价小结：上述回答区分了保密、认证与公开验证，也说明了可信公钥和防重放的必要性。关键是不能把“验签成功”直接等同于“现实身份可信、业务请求应被执行”，需要分别检查这些条件。

概念二：RSA-PSS 为什么同一消息能有不同签名

提示词：我在学习 RSA-PSS 的随机盐及参数匹配，请你以苏格拉底的方式对我进行提问，一次一个问题，帮助我解释“两份签名不同却都能验签成功”。

AI 问题 1：同一把私钥对同一消息签名两次，结果不同，就一定是程序出错吗？

解答：不一定。PSS 在编码时加入随机盐，所以即使消息摘要相同，也可能得到不同的编码和签名。应分别验证两份签名，而不是只比较它们是否相同。

AI 问题 2：验证者不保存签名者当时的随机状态，为什么还能验签？

解答：因为验签不是重新签一次。验证者通过公钥运算获得待检查的编码，按规则恢复并检查盐和摘要的关系，再与当前消息计算出的结果核对，因此不需要知道签名者当时的随机状态。

AI 问题 3：签名时使用 32 字节盐，而验证端强制要求 20 字节盐，应当怎样处理？

解答：应该拒绝，因为这份签名不符合验证端要求的参数。本次实验中的 `expected: 20 retrieved: 32` 就对应这种情况。应核对协议要求并统一参数，不能为了通过验证就忽略错误。

AI 问题 4：为什么不能让验签端也“重新签名”，再比较两个签名文件?

解答：首先，验签端不应该需要签名者的私钥；其次，随机化签名即使针对同一消息，结果也可能不同。因此必须调用公钥验签算法，不能通过重新签名后比较文件来代替验签。

AI 问题 5：已经计算 `SHA256(M)`，再把摘要交给 `dgst -sha256 -sign`，是否仍然等同于直接签名 M?

解答：不等同。这个命令会对输入再计算一次 SHA-256，实际处理的就成了摘要文件的摘要。需要先弄清接口接收的是原消息还是预计算摘要，不能把“签名会使用摘要”理解成“调用前一定要自己先算摘要”。

AI 评价小结：上述回答说明了随机化为什么不影响可验证性，也区分了 PSS 编码参数、验证策略与 API 输入层次。结合“错误参数失败，恢复正确参数成功”的实验，可以进一步理解为什么必须按协议配置参数，以及为什么不能用重新签名比较来代替验签。

五、学习实践过程遇到的问题与解决方式（2分）

1. 实践环境与结果

本节由 AI 助手实际执行：采用 Windows x64、GCC 11.5.0 和 OpenSSL 3.6.5（MSYS2 MinGW-w64 构建）。Python 仅用于组织命令、生成精确输入字节和保存日志，RSA 操作由 OpenSSL 与 C 程序完成。

最初复用的 OpenSSL 3.5.9 目录可以运行命令，但当前 `include/openssl` 下没有头文件，C 编译实际报错 `openssl/evp.h: No such file or directory`。核对后改用包含头文件与匹配 DLL 的 MinGW 开发包，再完成编译与全流程验证。首次日志保存在 `evidence/first-attempt-missing-headers.txt`。

下面的签名失败案例均为**主动构造的复现验证**，不是假定本人过去遇到过的故障。最终日志为 `evidence/experiment-log.txt`，共 **14 项检查通过**。

验证内容	实际结果
同一消息两次 RSA-PSS 签名	两份签名不同，均 `Verified OK`
RSA-2048 签名长度	256 字节
改消息内容、改换行、改签名、换公钥	均验签失败
将 PSS 签名按 PKCS#1 v1.5 验证	失败
强制错误盐长度 20，再恢复为 32	先失败，后成功

验证内容	实际结果
先计算摘要，再经 dgst 签名	对原消息验签失败，对摘要文件验签成功
PKCS#1 v1.5 重复签名	签名相同，且正常验签
C 签名交给命令行验证	`Verified OK`
命令行签名交给 C 验证	`VERIFY OK`
C 验证篡改后的消息	`VERIFY INVALID`，退出码 1

2. 问题一：文本看起来相同，验签却失败

现象（复现验证）：使用原始 `message.txt` 验签成功；把其中两处 LF 换行改为 CRLF，生成 `crlf.txt`，验签失败。

AI 排查思路：先比较实际字节，再检查公钥、签名和算法参数。不能仅凭编辑器里显示的文本一致就认定输入相同。

处理与验证：生成输入时明确编码和换行；C 端以 `rb` 方式读取，按 `fread` 返回的实际长度调用 Update。保持签名和参数不变，改回原始文件再验证。

```
原消息：44 字节，LF 换行        → Verified OK
修改后：46 字节，CRLF 换行      → Verification failure
```

核心经验：签名认证的是确定的字节序列。若业务需要规范化，应在签名前统一规则，验签端按同一协议处理。

3. 问题二：同为 RSA，为什么换一个填充参数就验签失败

现象（复现验证）：PSS 签名用 `rsa_padding_mode:pkcs1` 验证，提示填充检查失败；恢复 PSS，但把要求的盐长度设成 20，出现：

```
salt length check failed
expected: 20 retrieved: 32
```

AI 排查思路：RSA 是底层算法家族，完整签名方案还包括摘要、编码、MGF1 和盐长度策略。确认参数组合，不能只检查“密钥是不是 RSA”。

处理与验证：签名与验签统一为下面的参数，重新验证原签名成功：

```
rsa_padding_mode:pss
rsa_mgf1_md:sha256
rsa_pss_saltlen:32
消息摘要：SHA-256
```

核心经验：参数是协议的一部分。本例显式要求 32 字节盐；其他协议可能有不同的允许策略，应遵循协议而不是任意放宽检查。

4. 问题三：“先摘要再签名”导致重复摘要

现象（复现验证）：先生成 `digest.bin=SHA256(message.txt)`，然后用 `dgst -sha256 -sign` 对 `digest.bin` 签名。得到的签名对原始 `message.txt` 验证失败，却对 `digest.bin` 验证成功。

AI 排查思路：区分“签名算法内部会使用摘要”与“调用者必须先算好摘要”。前者不意味着所有接口都接收摘要作为输入。

```
# 第一次摘要
openssl dgst -sha256 -binary -out digest.bin message.txt

# 该命令又对 digest.bin 计算摘要
openssl dgst -sha256 -sign private.pem -sigopt rsa_padding_mode:pss -sigopt rsa_pss_saltlen:32 -s
igopt rsa_mgf1_md:sha256 -out double-hash.bin digest.bin
```

处理与验证：对于本例的 `dgst` 和 `EVP_DigestSign*`，直接传入原始消息。若确实使用接收预计算摘要的接口，应核对接口契约并正确配置算法，不能混用两条流程。

核心经验：先确认接口的输入层次，再实现数学流程；否则每一步可能都返回成功，最后的签名对象却不符合预期。

5. C 语言实现与交叉验证

源码 `code/rsa_pss.c` 提供 `sign` 和 `verify` 两个子命令。C 端将消息每 7 字节分块输入，实际输出为：

```
SIGN OK: message=44 bytes, signature=256 bytes
Verified OK          (OpenSSL 命令行验证 C 签名)
VERIFY OK            (C 验证 OpenSSL 命令行签名)
VERIFY INVALID       (C 验证篡改后的消息，退出码 1)
ALL 14 CHECKS PASSED
```

Linux 中安装相应编译器与 OpenSSL 开发包后，可按下面的方式编译；这是复现说明，本次实际编译环境为 Windows MinGW：

```
gcc -std=c11 -Wall -Wextra -Werror code/rsa_pss.c -o rsa_pss -lcrypto
./rsa_pss sign private.pem message.txt c-signature.bin
./rsa_pss verify public.pem message.txt c-signature.bin
```

保留的 `evidence/public.pem`、`message.txt`、`pss1.bin` 与 `c-signature.bin` 可以用于复核，无需私钥。临时测试私钥已随工作目录清理。

本周归纳：数字签名不仅是一次私钥运算，还包含安全的摘要与编码、可信公钥、明确的输入字节和严格的错误处理。实践中应同时检查正常路径和拒绝路径。

六、参考资料

- **AI 工具**：OpenCode，本次模型为 gpt-6-astra，用于知识整理、学习示例、程序编写及实际验证。
- **课程要求**：用户提供的《预习作业要求 (3).md》，第五周指定第 7 章，报告内容参考第一周评分模板。
- **教材目录**：《Windows C/C++加密解密实战》，朱晨冰、李建英，清华大学出版社，第 7 章；本次依据公开目录定位 7.1—7.5。
- **课程导图入口**：课程 mindmap。
- **官方命令文档**：OpenSSL dgst，签名验签参数、输入输出与编码注意事项。
- **官方接口文档**：EVP_DigestSignInit、EVP_DigestVerifyInit，流式签名验签、返回值与上下文资源管理。
- **实验工具来源**：MSYS2 OpenSSL 软件包，本次使用 3.6.5-1 的 MinGW-w64 开发文件与运行库。

作业提交说明（1分）

1. Markdown：`20241328蔡贸俊《密码系统设计》.md`。
2. PDF：`20241328蔡贸俊《密码系统设计》第5周.pdf`。
3. 导图：正文含 Mermaid 代码，`assets/week5-mindmap.png` 为渲染截图，移动 Markdown 时保留 `assets` 文件夹。
4. 附件：`code/` 为代码和复现脚本，`evidence/` 为原始日志、验证数据及检查结果。
5. 可选代码托管链接：20241328蔡贸俊 GitHub 仓库。本次交付文件位于本地第五周目录。